

Desarrollo Backend III

Informe técnico · Evaluación Final Transversal

Desarrollo Backend Avanzado: Spring Cloud y Batch

Estudiante	Jean Luc Santoni Camilo
Carrera	Analista Programador Computacional
Asignatura	Desarrollo Backend III (PBY2203)
Proyecto	Modernización del sistema legacy del Banco XYZ
Repositorio	github.com/JeanSantoni02/backend3
Fecha	Octubre de 2026

Índice

1. Introducción	4
1.1 Contexto	4
1.2 Objetivo	4
1.3 Alcance	4
2. Arquitectura de la solución	5
2.1 Vista general	5
2.2 Componentes	6
2.3 Tecnologías	6
2.4 Casos de uso	7
2.5 Estructura interna de un microservicio	7
3. Parte 1: Migración de procesos batch	9
3.1 Procesos migrados	9
3.2 Diseño de los jobs	9
3.3 Calidad de los datos	10
3.4 Manejo de excepciones y reintentos	11
3.5 Finalización y reejecución automática	11
3.6 Optimización del procesamiento	12
3.7 Integridad y equivalencia de los resultados	12
3.8 Resultados	13
4. Parte 2: Backend for Frontend	14
4.1 El problema y la solución	14
4.2 Autenticación y autorización por canal	14
4.3 Comunicaciones seguras	15
4.4 Independencia de los BFF	15
5. Parte 3: Microservicios con Spring Cloud	16
5.1 Servicios de dominio	16
5.2 Configuración centralizada	17
5.3 Descubrimiento, balanceo y enrutamiento	17
5.4 Seguridad con OAuth2	18
5.5 Tolerancia a fallos	19
5.6 Arquitectura orientada a eventos	20
5.7 Consistencia de datos en un entorno distribuido	21
5.8 Monitoreo y registro	22
5.9 Contenedores y orquestación	22

5.10 Preparación para la nube y escalado horizontal	3
6. Resultados obtenidos	3
6.1 Pruebas	3
6.2 Evidencia de ejecución	3
6.3 Comparación con el sistema legacy	3
7. Desafíos enfrentados y soluciones	3
8. Propuestas de mejora y próximos pasos	3
9. Conclusiones	3
Anexo A. Capturas de la plataforma en ejecución	3
Anexo B. Entregables	3

1. Introducción

1.1 Contexto

El Banco XYZ lleva más de treinta años operando sobre una plataforma legacy basada en COBOL y scripts Shell, ejecutada en servidores mainframe. Sobre ella corren procesos críticos: el procesamiento de transacciones, la generación de estados de cuenta, el cálculo de intereses y la gestión de clientes.

Esa plataforma tiene tres problemas que el banco ya no puede sostener: escala solo agregando capacidad al mismo equipo, su mantenimiento es caro y escaso en especialistas, y conectarla con canales nuevos obliga a pasar por un backend monolítico que entrega los mismos datos a todos. Además, una falla en un módulo puede detener al resto, y la seguridad está centralizada en un solo punto.

1.2 Objetivo

Migrar el sistema a una arquitectura de microservicios preparada para la nube, que permita escalar cada parte por separado, tolerar fallas sin detener la operación, proteger cada servicio de forma independiente y sumar canales o consumidores nuevos sin modificar lo que ya funciona.

1.3 Alcance

Tabla 1: Partes del proyecto y lo que se construyó en cada una

Parte	Qué se construyó
1. Migración batch	Tres jobs de Spring Batch que reemplazan los procesos legacy de transacciones diarias, intereses y estados de cuenta anuales, con manejo de errores, reintentos, reejecución automática y procesamiento en paralelo.
2. Backend for Frontend	Un backend por canal (web, móvil y cajero automático), cada uno con su propio modelo de respuesta y su propia autenticación.
3. Microservicios	Servicios de gestión de clientes, gestión de cuentas y procesamiento de pagos, sobre una plataforma con configuración centralizada, descubrimiento, gateway con balanceo de carga, OAuth2, tolerancia a fallos, eventos y contenedores.

Quedan fuera del alcance las interfaces de usuario de cada canal, que se simulan con llamadas HTTP, y el despliegue efectivo en una cuenta de AWS: el procedimiento queda documentado paso a paso en `despliegue.md` y las imágenes Docker están listas para publicarse.

2. Arquitectura de la solución

2.1 Vista general

La solución se organiza en capas. Los canales nunca hablan directamente con los servicios: toda petición entra por un único gateway que termina el cifrado TLS, decide a qué servicio va según la ruta y reparte la carga entre las instancias disponibles. Detrás están los tres BFF, que atienden a cada canal, y los servicios de dominio, que concentran las reglas del negocio. Los datos viven en PostgreSQL y los eventos viajan por un broker de mensajería.

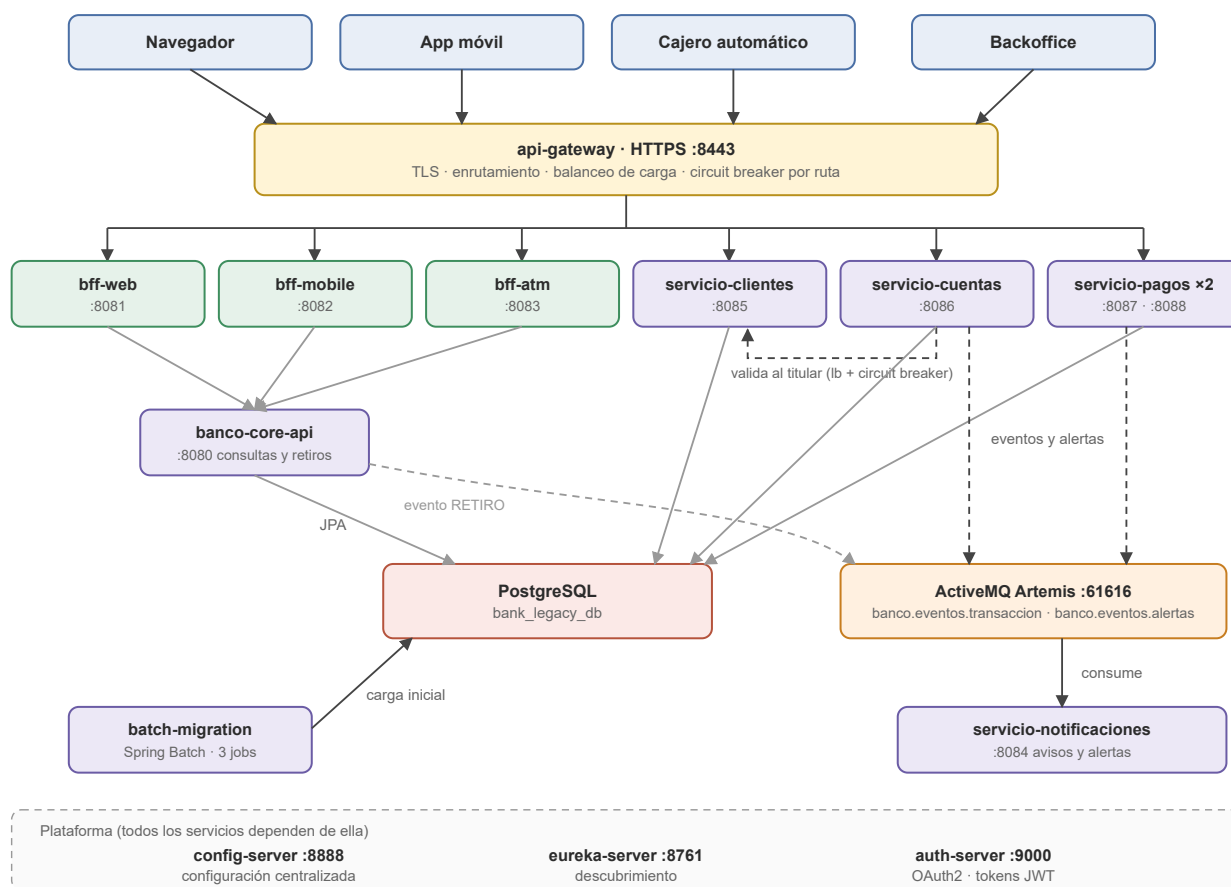


Figura 1. Arquitectura general de la plataforma.

Tres servicios de plataforma sostienen al resto. El **config-server** entrega a cada servicio su configuración al arrancar; el **eureka-server** mantiene el registro de qué instancias están vivas y en qué dirección; y el **auth-server** emite los tokens que los servicios exigen para responder.

2.2 Componentes

Tabla 2: Módulos del proyecto

Módulo	Puerto	Responsabilidad
batch-migration	—	Carga y validación de los archivos legacy en tres jobs
config-server	8888	Configuración centralizada de todos los servicios
eureka-server	8761	Registro y descubrimiento de servicios
auth-server	9000	Servidor de autorización OAuth2: emite tokens JWT
api-gateway	8443	Entrada única por HTTPS, enrutamiento, balanceo y circuit breaker por ruta
servicio-clientes	8085	Información personal, perfil y estado de los clientes
servicio-cuentas	8086	Apertura, mantenimiento y cierre de cuentas
servicio-pagos	8087, 8088	Depósitos, transferencias y pagos; publica eventos y alertas
servicio-notificaciones	8084	Consume eventos y alertas y genera los avisos
banco-core-api	8080	Consultas sobre los datos migrados y retiros de cajero
bff-web	8081	Backend del portal: respuestas completas
bff-mobile	8082	Backend de la app: respuestas livianas
bff-atm	8083	Backend de cajeros: operaciones críticas con tolerancia a fallos

2.3 Tecnologías

Tabla 3: Tecnologías utilizadas

Tecnología	Versión	Uso
Java	17	Lenguaje de todos los servicios
Spring Boot	3.2	Base de cada microservicio
Spring Batch	5	Migración de los procesos legacy
Spring Cloud	2023.0	Config Server, Eureka, Gateway, LoadBalancer
Spring Authorization Server	1.2	Emisión de tokens OAuth2
Spring Security	6.2	Validación de JWT y autorización por scope
Resilience4j	2.1	Circuit breaker, reintentos y límites de tiempo
ActiveMQ Artemis	2.31	Broker JMS para eventos y alertas
PostgreSQL	16 / 18	Base de datos relacional

Tecnología	Versión	Uso
Micrometer	1.12	Métricas Prometheus y traceld en los logs
Docker y Docker Compose	Compose v2	Imágenes y orquestación
springdoc OpenAPI	2.3	Documentación interactiva de cada API
JUnit 5 y Mockito	—	Pruebas unitarias

2.4 Casos de uso

El sistema atiende a cinco tipos de actor. Los clientes consultan desde la web o la app; el usuario de cajero consulta saldo y retira efectivo; el operador de backoffice registra clientes, administra cuentas y ejecuta pagos; el planificador lanza los procesos batch; y el equipo de seguridad revisa las alertas y sigue el recorrido de una petición entre servicios.

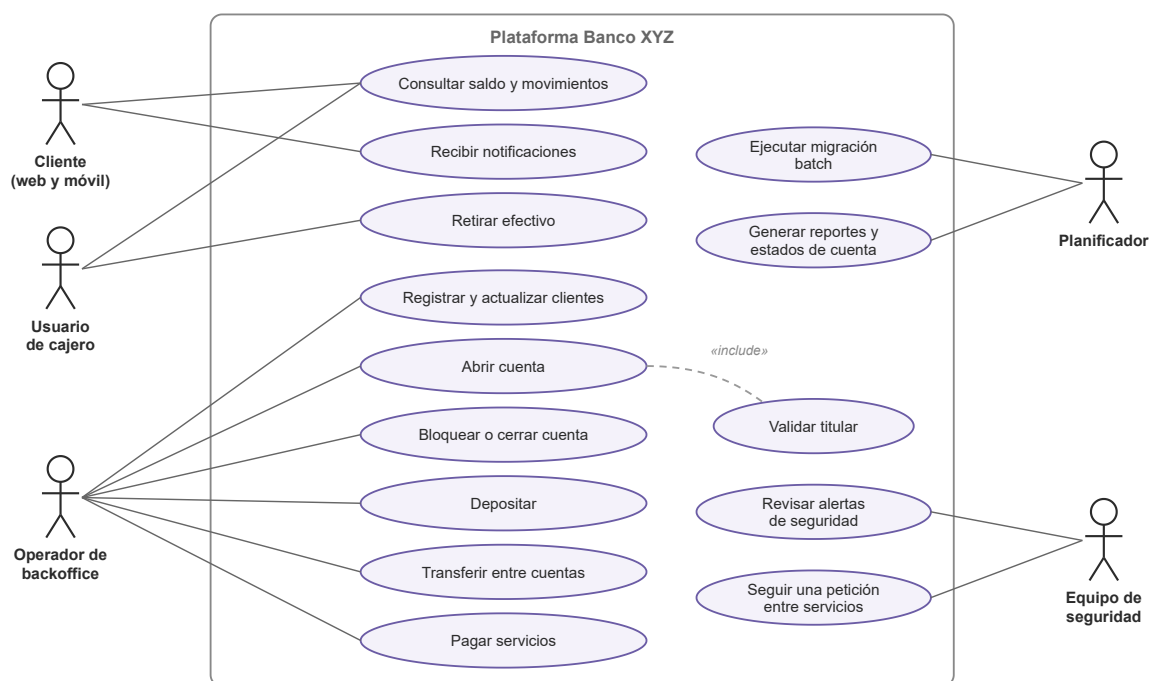


Figura 2. Diagrama de casos de uso.

2.5 Estructura interna de un microservicio

Los servicios de dominio comparten la misma organización interna. La figura usa como ejemplo a **servicio-cuentas**, que además de su base de datos depende de otro servicio y del broker.

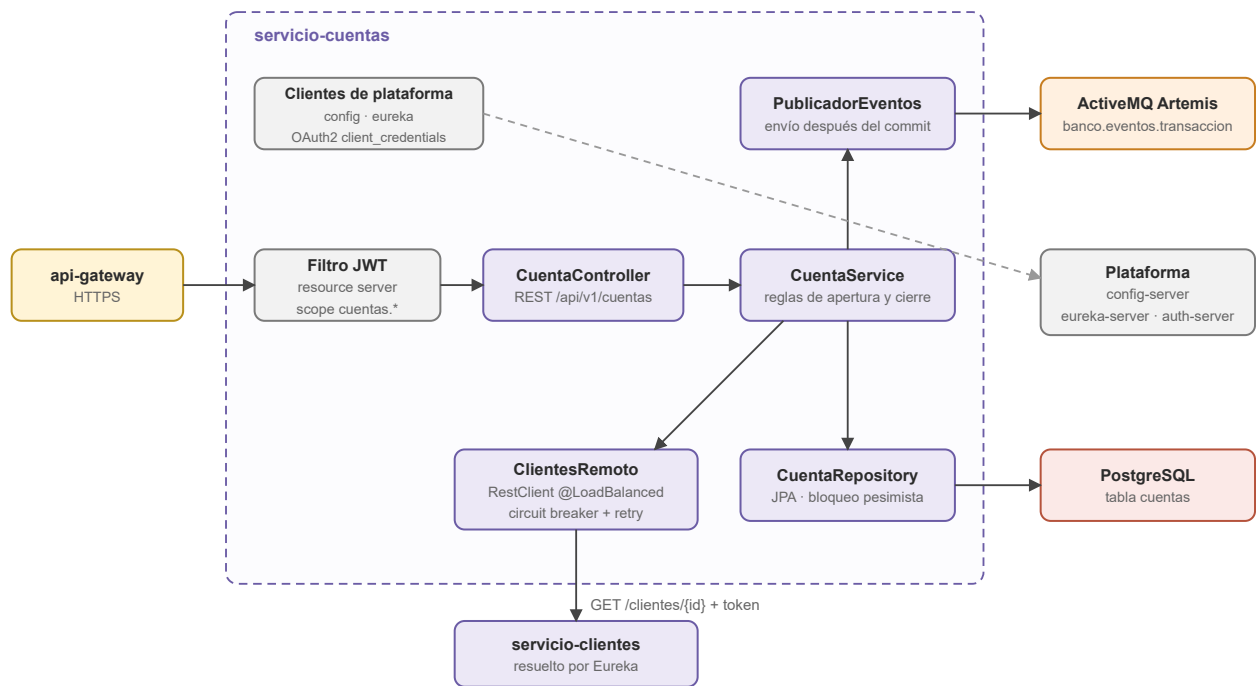


Figura 3. Componentes internos de servicio-cuentas y sus dependencias.

Cada petición atraviesa primero el filtro JWT, que rechaza con 401 lo que no trae token y con 403 lo que trae un token sin el scope necesario. El controlador solo traduce HTTP; las reglas viven en el servicio. La llamada a servicio-clientes pasa por un componente aparte, **CientesRemoto**, que concentra el circuit breaker, los reintentos y el comportamiento alternativo, de modo que el servicio no mezcla reglas de negocio con manejo de fallas de red.

3. Parte 1: Migración de procesos batch

3.1 Procesos migrados

Los datos de entrada son los archivos oficiales del repositorio `fin_legacy_data`, con 1.000 filas cada uno.

Tabla 4: Correspondencia entre los procesos legacy y los jobs

Proceso legacy	Job	Archivo	Resultado
Reporte de transacciones diarias	jobReporteTransaccionesDiarias	movimientos_financieros_diarios.csv	Detalle validado y resumen por día con anomalías
Cálculo de intereses	jobCalculoInteresesMensuales	intereses_trimestrales.csv	Interés por registro y saldo consolidado por cuenta
Estados de cuenta anuales	jobEstadosCuentaAnuales	estados_financieros_anuales.csv	Movimientos normalizados y estado por cuenta y año

3.2 Diseño de los jobs

Los tres jobs siguen la misma forma: un step que limpia el resultado anterior, un step de carga particionado que lee, procesa y escribe en paralelo, y un step final de agregación que construye el resumen con una sola sentencia `INSERT ... SELECT`. El job de intereses agrega un step intermedio que descarta los registros duplicados.

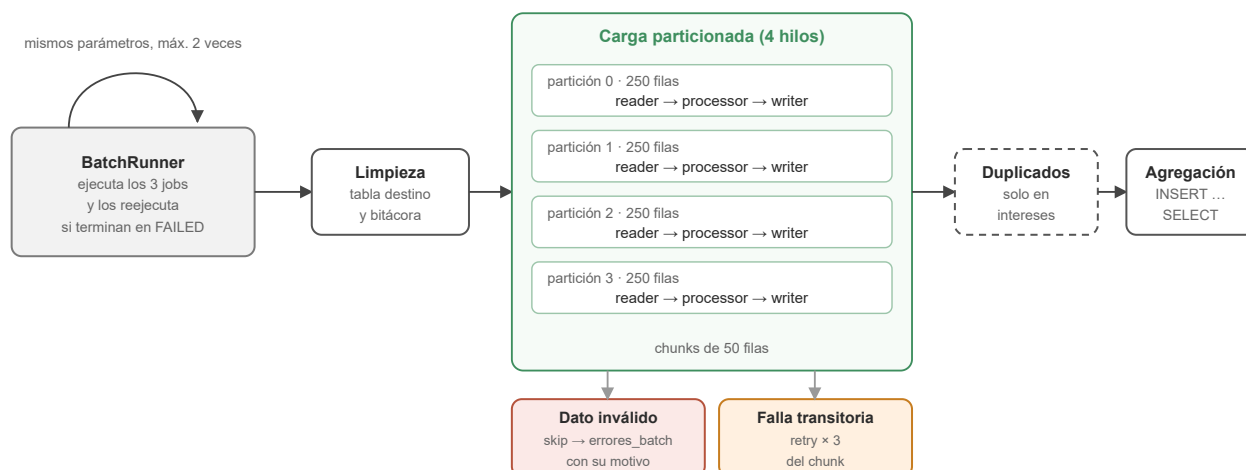


Figura 4. Estructura de los jobs y tratamiento de errores.

3.3 Calidad de los datos

Los archivos legacy traen errores intencionales. Se perfilaron antes de escribir las reglas, para que cada decisión respondiera a un caso real del archivo y no a uno supuesto.

Tabla 5: Errores encontrados en los archivos de entrada

Archivo	Problema	Filas
Movimientos diarios	Cuatro formatos de fecha mezclados (yyyy-MM-dd, yyyy/MM/dd, dd/MM/yyyy, dd-MM-yyyy)	1.000
Movimientos diarios	Mes 13 inexistente (2024-13-01)	49
Movimientos diarios	Monto vacío / negativo / cero	168 / 125 / 20
Movimientos diarios	Tipo invalido o desconocido	317 / 66
Intereses	Tipo de cuenta -1 o unknown	281 / 44
Intereses	Saldo vacío	204
Intereses	Edad vacía o fuera de rango (150)	171 / 46
Intereses	Filas idénticas a otra anterior	7
Estados anuales	Tipo escrito con y sin tilde (deposito, depósito)	293 / 51
Estados anuales	Monto vacío	48

Cada registro recibe uno de tres tratamientos explícitos:

- **Válido o corregido.** Se normaliza y se guarda. Las fechas se leen con los cuatro formatos en modo estricto, de manera que el mes 13 falla en lugar de convertirse en silencio en

diciembre; los textos se pasan a minúsculas y sin tildes, con lo que *depósito* y *deposito* quedan unificados.

- **Anomalía.** Se guarda, marcado para revisión. Un monto negativo o un tipo fuera de catálogo en una transacción no se descarta, porque detectar anomalías es justamente el objetivo del reporte diario.
- **Descartado.** Se omite y queda registrado en la tabla `errores_batch` con el job, el step, el motivo y la fila original. Se descarta solo lo que no puede cumplir su función: una transacción sin fecha válida no se puede asignar a ningún día, y un interés sin saldo o sin tipo de cuenta no tiene sobre qué calcularse.

3.4 Manejo de excepciones y reintentos

La política de omisión distingue el origen del error, porque un dato sucio y una base de datos caída no se tratan igual:

Tabla 6: Política de tolerancia a fallos del batch

Situación	Decisión	Motivo
Registro inválido	Se omite, hasta 1.000 por step	Es un dato sucio conocido del legacy; abortar impediría que la migración terminara
Línea malformada	Se omite, hasta 50	Muchas líneas malformadas indican un archivo corrupto: conviene detenerse
Falla transitoria de la base	Se reintenta el chunk hasta 3 veces	El registro es válido; perderlo sería perder datos de dinero
Falla persistente	El job termina en FAILED y se reejecuta	Ver sección 3.5

La bitácora de errores se escribe en una transacción independiente (`REQUIRES_NEW`). Cuando Spring Batch omite un registro, la transacción del chunk ya está marcada para deshacerse; si el error se anotara en ella, se perdería junto con el chunk.

3.5 Finalización y reejecución automática

Si un job termina en estado FAILED porque la falla duró más que los reintentos, el lanzador lo vuelve a ejecutar automáticamente con **los mismos parámetros**, hasta dos veces y con una espera que se duplica en cada intento (2 y 4 segundos).

Usar los mismos parámetros es lo que convierte el reintento en una reanudación: Spring Batch no crea una ejecución nueva, sino que retoma la instancia fallida. Los steps que ya habían terminado no se repiten, y el que falló continúa desde el último chunk confirmado. Para que un problema permanente no se reintente para siempre, los steps de carga declaran un

`startLimit` igual al intento original más las reejecuciones; agotado ese límite, el proceso termina con código de salida 1 para que el planificador lo detecte.

```
JobExecution ejecucion = jobLauncher.run(job, parametros);
for (int intento = 1; ejecucion.getStatus() == BatchStatus.FAILED
    && intento <= propiedades.getReejecucionesMaximas(); intento++) {
    Thread.sleep(espera);
    espera *= 2;
    ejecucion = jobLauncher.run(job, parametros);    // mismos parametros: reanuda
}
```

Una prueba unitaria verifica los tres casos: el job que se recupera en la reejecución, el que agota los intentos y el que termina bien al primer intento.

3.6 Optimización del procesamiento

El step de carga se divide en cuatro particiones que procesan rangos disjuntos del archivo en hilos distintos, con chunks de 50 filas. El número de particiones se cambia por configuración, sin recompilar. Medido sobre `fin_legacy_data`, tomando el mejor de dos tiempos por configuración:

Tabla 7: Tiempo de cada job con una y con cuatro particiones

Job	1 partición	4 particiones	Mejora
Reporte de transacciones diarias	1.256 ms	952 ms	24 %
Cálculo de intereses	872 ms	509 ms	42 %
Estados de cuenta anuales	563 ms	250 ms	56 %
Total	2.691 ms	1.711 ms	36 %

Con 1.000 filas por archivo la ganancia está limitada por el costo fijo de cada job. El reparto en particiones crece con el volumen: el mismo diseño admite más hilos o, más adelante, particiones remotas en máquinas distintas.

3.7 Integridad y equivalencia de los resultados

No se cuenta con las salidas del sistema legacy para compararlas fila por fila, así que la equivalencia se verificó por tres vías independientes:

- **Cuadratura con el origen.** En cada archivo, las filas conservadas más las descartadas suman las 1.000 leídas, y cada descarte tiene su motivo registrado.
- **Coherencia interna.** El resumen diario suma exactamente el mismo importe que el detalle de transacciones (820.400), y los registros consolidados por cuenta son los mismos 538 intereses calculados.

- **Determinismo.** Las corridas con una y con cuatro particiones producen los mismos conteos y las mismas sumas de control en todas las tablas: procesar en paralelo no altera el resultado.

Durante la migración al dataset nuevo aparecieron dos problemas de integridad que se corrigieron. El job de transacciones no limpiaba su tabla antes de cargar y conservaba filas de una corrida anterior cuyo identificador no venía en el archivo nuevo: la tabla tenía 956 filas cuando el archivo aportaba 789, y el resumen diario las sumaba. Ahora los tres jobs recargan su resultado completo. El segundo problema eran los registros duplicados de intereses, que se tratan en un step propio después de la carga, porque las dos copias pueden caer en particiones distintas y ninguna partición ve a la otra.

3.8 Resultados

Tabla 8: Resultado de la migración sobre fin_legacy_data

Job	Leídas	Conservadas	Descartadas	Resultado agregado
Transacciones diarias	1.000	789 (392 válidas, 397 anomalías)	211	Resumen de 325 días
Intereses	1.000	538	462 (incluye 3 duplicados)	50 cuentas consolidadas
Estados anuales	1.000	952	48	20 estados de cuenta

Tabla 9: Motivos de descarte registrados en errores_batch

Job	Motivo	Filas
Transacciones	Monto ausente o no numérico	162
Transacciones	Fecha inválida para un reporte diario	49
Intereses	Tipo de cuenta no reconocido (-1)	219
Intereses	Saldo ausente	204
Intereses	Tipo de cuenta no reconocido (unknown)	36
Intereses	Registro duplicado	3
Estados anuales	Monto ausente o no numérico	48
Total		721

4. Parte 2: Backend for Frontend

4.1 El problema y la solución

Con un backend único, los tres canales reciben las mismas respuestas. La app móvil descarga datos que no muestra; el portal necesita varias llamadas para armar una pantalla; y la operación de retiro queda expuesta en el mismo punto que usa un navegador. Cualquier cambio para favorecer a un canal afecta a los otros dos.

El patrón BFF resuelve esto con un backend dedicado por canal. Los tres consumen el mismo servicio de dominio, pero cada uno decide qué datos entrega, en qué forma y con qué controles.

Tabla 10: Los tres BFF

BFF	Endpoints	Lo que lo distingue
bff-web	/bff/web/cuentas /bff/web/cuentas/{id}/panel	Arma la pantalla completa en una sola llamada: saldo, intereses, estado anual e historial paginado
bff-mobile	/bff/movil/cuentas/{id} /bff/movil/cuentas/{id}/saldo	Solo lo esencial: para la misma cuenta transmite 418 bytes contra 3.826 del web, un 89 % menos
bff-atm	/bff/atm/cuentas/{id}/saldo /bff/atm/cuentas/{id}/retiro	Superficie mínima, montos múltiplos de 1.000 hasta 200.000, idempotencia por referencia y circuit breaker

4.2 Autenticación y autorización por canal

Cada BFF es un cliente distinto del servidor de autorización y obtiene su propio token con el flujo *client_credentials*: el que llama al dominio es una máquina, no una persona. Cada cliente tiene solo los permisos (scopes) que su canal necesita:

Tabla 11: Clientes OAuth2 y sus permisos

Cliente	Scopes	Qué puede hacer
bff-web	cuentas.leer, transacciones.leer	Solo consultas
bff-mobile	cuentas.leer	Solo consultas de cuentas
bff-atm	cuentas.leer, retiros.escribir	El único canal que puede retirar
portal-operaciones	clientes, cuentas y pagos (lectura y escritura)	Backoffice de operaciones
servicio-cuentas	clientes.leer	Consultar al titular al abrir una cuenta

El endpoint de retiro exige el scope `retiros.escribir`. Aunque el BFF móvil fuera comprometido y su atacante conociera la ruta, su token no le permitiría retirar: el daño queda acotado a lo que ese canal ya podía hacer. El BFF de cajeros suma una segunda capa: cada terminal se identifica con su clave, que se compara en tiempo constante para no filtrar información por la duración de la respuesta.

4.3 Comunicaciones seguras

Todo el tráfico externo entra por el gateway, que solo atiende HTTPS. La conexión verificada negocia TLS 1.3 con cifrado AES de 128 bits, y el certificado declara los nombres `localhost`, `api-gateway` y `127.0.0.1`. Una petición en HTTP plano al mismo puerto no recibe respuesta.

El certificado de desarrollo es autofirmado. En la nube lo reemplaza uno emitido por AWS Certificate Manager en el balanceador, que vuelve a cifrar el tráfico hacia el gateway, de modo que nunca viaja en claro ni siquiera dentro de la red privada. Los tokens son JWT firmados con RSA y vencen a los 30 minutos.

4.4 Independencia de los BFF

Cada BFF es un proyecto con su propio despliegue, su propia configuración en el config-server y su propio registro en Eureka. Un equipo puede cambiar el BFF móvil sin tocar el web ni el de cajeros, y sin esperar cambios en el servicio de dominio, que expone una API genérica y estable.

5. Parte 3: Microservicios con Spring Cloud

5.1 Servicios de dominio

Tabla 12: Endpoints de los servicios de dominio

Servicio	Endpoint	Función
Cientes	GET /api/v1/clientes	Lista, con filtro por nombre
Cientes	POST /api/v1/clientes	Alta con RUT validado por dígito verificador
Cientes	PUT /api/v1/clientes/{id}	Actualización del perfil
Cientes	PATCH /api/v1/clientes/{id}/estado	Activación o inactivación
Cuentas	POST /api/v1/cuentas	Apertura, validando al titular
Cuentas	PATCH /api/v1/cuentas/{id}	Mantenimiento: tipo, bloqueo y desbloqueo
Cuentas	POST /api/v1/cuentas/{id}/cierre	Cierre, solo con saldo cero
Pagos	POST /api/v1/pagos/depositos	Depósito
Pagos	POST /api/v1/pagos/transferencias	Transferencia entre cuentas
Pagos	POST /api/v1/pagos/servicios	Pago de un servicio o convenio
Pagos	GET /api/v1/pagos/{referencia}	Consulta de una operación

Gestión de clientes

Valida el RUT con su dígito verificador calculado por módulo 11, rechaza con 409 un RUT ya registrado y con 400 uno mal formado. El RUT identifica al cliente y no se modifica al actualizar el perfil.

Gestión de cuentas

Antes de abrir una cuenta consulta al titular en servicio-clientes. Si no existe responde 422, si está inactivo también, y si el servicio de clientes no contesta responde 503 sin abrir nada: una cuenta abierta sin confirmar a su titular podría quedar huérfana. Las cuentas abiertas en línea se numeran desde 100.000, rango que el batch nunca toca cuando reconstruye las cuentas migradas. El cierre exige saldo cero y bloquea la fila mientras decide, para que un pago concurrente no cambie el saldo en ese instante.

Procesamiento de pagos

Aplica depósitos, transferencias y pagos. Cada operación lleva una referencia que la hace idempotente: repetir una petición con la misma referencia devuelve la operación original con `duplicado: true` y no vuelve a mover dinero. Rechaza con 422 las operaciones sin saldo

suficiente y las que apuntan a una cuenta bloqueada o cerrada; estas últimas generan además una alerta de seguridad.

5.2 Configuración centralizada

Ningún servicio guarda su configuración de negocio en su propio paquete. Al arrancar, cada uno pide al config-server su archivo y el archivo común. Los `application.yml` locales tienen solo tres datos: el nombre del servicio, su puerto y la dirección del config-server.

Tabla 13: Archivos del repositorio de configuración

Archivo	Contenido
application.yml	Común a todos: Eureka, actuador, trazas, formato de log y nombres de los tópicos
api-gateway.yml	Rutas, balanceo y circuit breaker por ruta
servicio-cuentas.yml	Base de datos, broker y política de resiliencia hacia clientes
servicio-pagos.yml	Base de datos, broker y umbral de alerta
bff-atm.yml	Umbral del circuit breaker, reintentos y límites del cajero
banco-core-api.yml, servicio-clientes.yml, servicio-notificaciones.yml	Datos de conexión de cada uno

En desarrollo el config-server lee de una carpeta (perfil *native*). En la nube pasa a leer de un repositorio Git con dos variables de entorno, sin cambiar el código: modificar un umbral se convierte en un commit.

5.3 Descubrimiento, balanceo y enrutamiento

Cada instancia se registra en Eureka al arrancar y renueva su registro cada pocos segundos. El gateway define sus rutas con el prefijo `lb://`: en lugar de una dirección fija, usa el nombre del servicio, consulta a Eureka qué instancias existen y reparte las peticiones entre ellas por turnos (round robin). Lo mismo hace servicio-cuentas al llamar a servicio-clientes, con un `RestClient` marcado como `@LoadBalanced`.

Tabla 14: Rutas del gateway

Ruta	Destino	Circuit breaker
/api/v1/clientes/**	lb://servicio-clientes	Sí
/api/v1/cuentas/**	lb://servicio-cuentas	Sí
/api/v1/pagos/**	lb://servicio-pagos	Sí
/api/v1/notificaciones/**, /api/v1/alertas/**	lb://servicio-notificaciones	No

Ruta	Destino	Circuit breaker
/bff/web/**, /bff/movil/**	lb://bff-web, lb://bff-mobile	Sí
/bff/atm/**	lb://bff-atm	No: el BFF tiene su propia política

Para hacer visible el balanceo, servicio-pagos agrega a cada respuesta la cabecera `x-Atendido-Por` con la instancia que la atendió. Con dos instancias levantadas, seis consultas seguidas al mismo endpoint se alternaron entre los puertos 8088 y 8087. Una tercera instancia empezaría a recibir tráfico apenas se registrara, sin tocar ninguna configuración.

```

peticion 1 -> LAPTOP-2RGGN8RD:8088
peticion 2 -> LAPTOP-2RGGN8RD:8087
peticion 3 -> LAPTOP-2RGGN8RD:8088
peticion 4 -> LAPTOP-2RGGN8RD:8087
peticion 5 -> LAPTOP-2RGGN8RD:8088
peticion 6 -> LAPTOP-2RGGN8RD:8087

```

5.4 Seguridad con OAuth2

Todos los servicios de dominio son *resource servers*: validan la firma del JWT con la clave pública del servidor de autorización, que descargan una vez y guardan, de modo que no consultan al auth-server en cada petición. Luego convierten el claim `scope` en permisos y autorizan cada endpoint según el método HTTP: las lecturas exigen `*.leer` y las escrituras `*.escribir`.

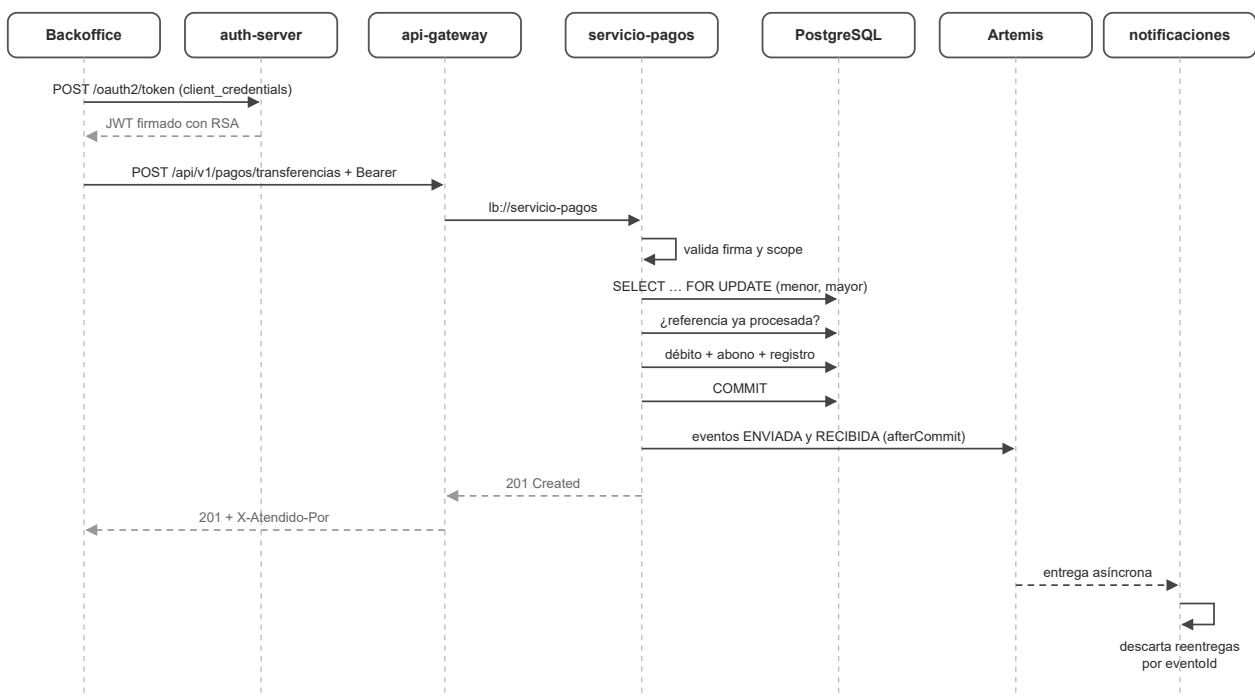


Figura 5. Secuencia de una transferencia: token, enrutamiento, persistencia y evento.

Cuando un servicio llama a otro también se autentica. servicio-cuentas tiene su propio cliente OAuth2 con un único permiso, `clientes.leer`: puede consultar a un titular pero no modificarlo. El token se pide una vez y se reutiliza hasta que vence.

Tabla 15: Respuestas de seguridad verificadas a través del gateway

Prueba	Resultado
Petición sin token	401 Unauthorized
Token del BFF móvil intentando depositar	403 Forbidden
Token del portal de operaciones	201 Created
Conexión TLS al gateway	TLS 1.3, AES 128 bits
Petición HTTP sin cifrar	Sin respuesta

5.5 Tolerancia a fallos

Las llamadas a servicios externos se protegen con Resilience4j en tres puntos: el BFF de cajeros hacia el dominio, servicio-cuentas hacia servicio-clientes, y el gateway en cada ruta. El circuit breaker cuenta las fallas sobre una ventana de llamadas; si superan el umbral, deja de intentar la llamada durante un tiempo y responde de inmediato con el comportamiento alternativo.

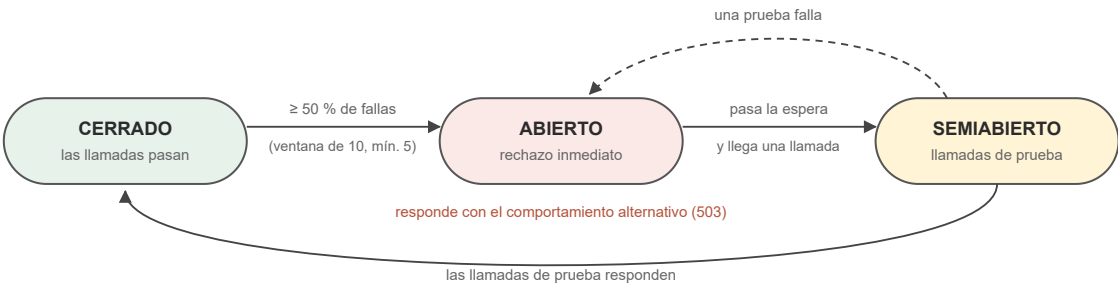


Figura 6. Estados del circuit breaker.

Tabla 16: Configuración de resiliencia

Punto	Circuit breaker	Reintentos	Comportamiento alternativo
bff-atm → dominio	50 % en 10 llamadas, abierto 20 s	3, desde 400 ms ×2	Saldo: 503. Retiro: queda indeterminado para conciliación
servicio-cuentas → clientes	50 % en 10 llamadas, abierto 15 s	3, desde 300 ms ×2	503 claro; la cuenta no se abre
gateway → cada servicio	50 % en 10 llamadas, abierto 15 s	—	503 con mensaje por servicio

Dos decisiones de diseño son las que hacen que el circuito se comporte bien:

- **Un rechazo de negocio no es una falla.** Saldo insuficiente o un cliente inexistente son respuestas correctas del servicio. Están declarados como excepciones ignoradas: si contaran, una tanda de clientes sin fondos abriría el circuito y dejaría sin servicio a los que sí tienen saldo.
- **El circuito envuelve al reintento.** El orden de los aspectos está fijado para que primero se reintente y solo la operación completa fallida cuente para el circuito. Con el orden por defecto ocurre lo inverso: una sola falla de red cuenta tres veces y abre el circuito antes de tiempo.

En la prueba, con servicio-clientes detenido, seis intentos de apertura respondieron 503 con el mensaje alternativo. El circuito contaba una apertura exitosa previa; con cuatro fallas la tasa llegó al 80 % y se abrió, y los dos intentos siguientes se rechazaron sin llamar al servicio caído. Cada apertura cuenta como una sola llamada aunque internamente se reintente tres veces. Al volver a levantar servicio-clientes, las aperturas se reanudaron.

```
circuito clientes: estado=OPEN fallas=4 tasa=80.0% rechazadas sin llamar=2
```

5.6 Arquitectura orientada a eventos

Se eligió el patrón de publicación y suscripción con notificación de eventos sobre JMS. Cada operación completada publica un evento con lo mínimo para que un consumidor actúe: qué pasó, en qué cuenta, por cuánto y con qué referencia. Las alertas de seguridad viajan por un tópico separado, para que un equipo de monitoreo pueda escucharlas sin recibir todas las transacciones.

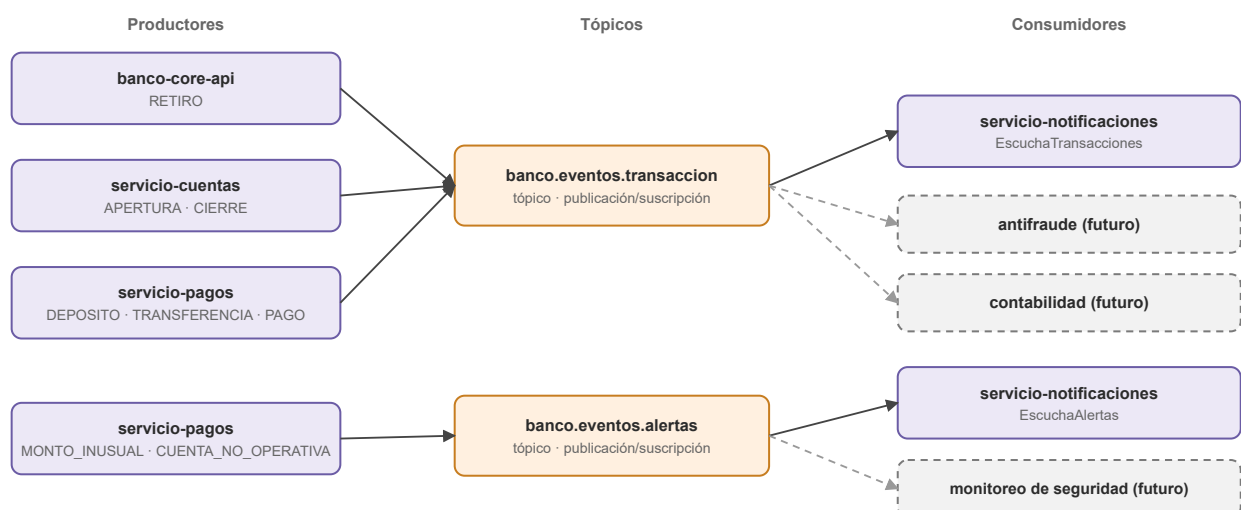


Figura 7. Productores, tópicos y consumidores.

Tabla 17: Eventos publicados

Tópico	Evento	Productor
banco.eventos.transaccion	RETIRO	banco-core-api
banco.eventos.transaccion	APERTURA, CIERRE	servicio-cuentas
banco.eventos.transaccion	DEPOSITO, TRANSFERENCIA_ENVIADA, TRANSFERENCIA_RECIBIDA, PAGO	servicio-pagos
banco.eventos.alertas	MONTO_INUSUAL (desde 1.000.000)	servicio-pagos
banco.eventos.alertas	CUENTA_NO_OPERATIVA	servicio-pagos

Se prefirió un tópico a una cola porque cada mensaje debe llegar a todos los interesados: hoy al servicio de notificaciones, mañana a un servicio antifraude o de contabilidad, que se suman sin tocar a los productores. Se eligió JMS sobre Kafka porque el volumen no requiere retener ni reprocesar el historial de eventos, y Artemis puede correr embebido en desarrollo.

5.7 Consistencia de datos en un entorno distribuido

- **Transferencias atómicas.** El débito y el abono ocurren en la misma transacción de base de datos: o se aplican los dos o ninguno.
- **Bloqueo ordenado.** Las dos cuentas se bloquean con `SELECT ... FOR UPDATE` siempre en orden ascendente. Si una transferencia va de A a B y otra de B a A al mismo tiempo, ambas piden primero la misma cuenta y una espera a la otra, en lugar de quedar bloqueadas mutuamente.
- **Idempotencia verificada después del bloqueo.** La comprobación de la referencia ocurre una vez tomado el bloqueo. Si dos peticiones idénticas llegan juntas, la segunda espera, y al continuar ya ve la operación confirmada por la primera.
- **Eventos después del commit.** El evento se publica recién cuando la transacción confirma. Si se publicara antes y la transacción se deshiciera, el cliente recibiría el aviso de un movimiento que nunca ocurrió.
- **Consumidores idempotentes.** El broker puede reentregar un mensaje. El consumidor guarda los identificadores ya procesados y descarta los repetidos: en la prueba, repetir una transferencia no cambió el contador de notificaciones.

Los servicios comparten la instancia de PostgreSQL, pero cada uno es dueño de sus operaciones: clientes tiene su propia tabla, cuentas gestiona el ciclo de vida y pagos mueve saldos. Es la etapa de transición habitual de una migración; separar una base por servicio queda como mejora en la sección 8.

5.8 Monitoreo y registro

Tabla 18: Mecanismos de observación

Mecanismo	Dónde	Para qué
Salud	/actuador/health en cada servicio	Healthchecks de Docker y del balanceador
Métricas	/actuador/prometheus	Peticiones por endpoint, tiempos, errores y estado de la JVM
Estado de resiliencia	/actuador/circuitbreakers	Estado del circuito, tasa de fallas y llamadas rechazadas
Trazas	traceld en cada línea de log	Seguir una petición de un servicio a otro
Auditoría de cajeros	Logger AUDITORIA_ATM	Registro de cada operación por terminal
Bitácora del batch	Tabla errores_batch	Qué registros no entraron y por qué

```
19:15:10.985 INFO [6ac580fe5ae2f37436078d9df92480b0] [io-8086-exec-2] CuentaService
: Cuenta abierta | cuenta=100000 cliente=6 tipo=ahorro deposito=50000
```

5.9 Contenedores y orquestación

Cada servicio tiene un Dockerfile en dos etapas: Maven compila en la primera y la segunda copia solo el jar sobre un JRE, con un usuario sin privilegios. La imagen final no lleva compilador ni código fuente, lo que reduce su tamaño y su superficie de ataque.

El `docker-compose.yml` orquesta quince contenedores: PostgreSQL, Artemis, los tres servicios de plataforma, el gateway, los cinco servicios de dominio y notificaciones (con dos réplicas de pagos) y los tres BFF. Cada dependencia espera a que la anterior esté sana con `condition: service_healthy`, no solo a que su contenedor haya arrancado. Los servicios de dominio no publican puertos: solo se llega a ellos a través del gateway.

La versión de diez contenedores se ejecutó en Docker durante la semana 8, con todos los servicios en estado *healthy* y el flujo completo funcionando entre contenedores. La versión final, con los cuatro servicios nuevos, quedó validada con `docker compose config`, y la plataforma completa se verificó en ejecución con sus trece procesos.

5.10 Preparación para la nube y escalado horizontal

El documento `despliegue.md` describe dos caminos en AWS: una instancia EC2 con el mismo compose, para demostración, y una arquitectura de producción sobre ECS Fargate con servicios administrados.

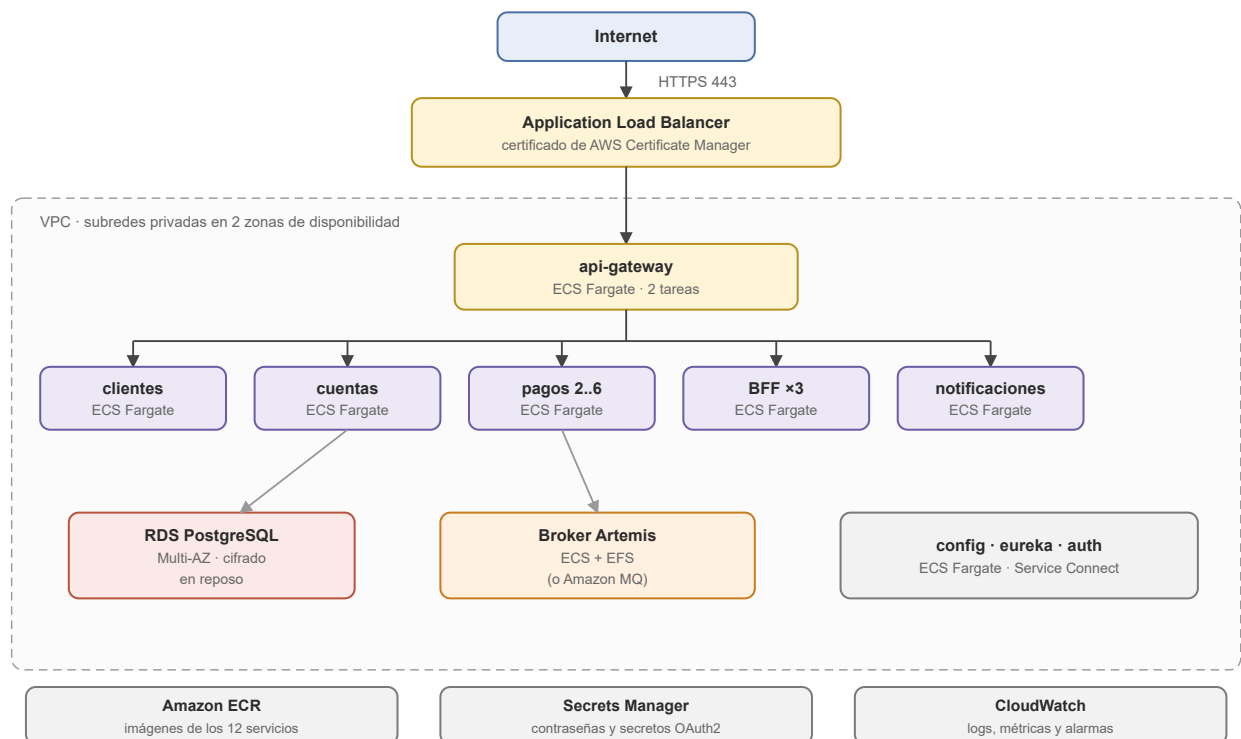


Figura 8. Arquitectura de despliegue en AWS.

El escalado horizontal no requiere cambios en el código. Una política de ECS agrega tareas de servicio-pagos cuando la CPU promedio supera el 60 %, entre dos y seis. Cada tarea nueva se registra sola en Eureka y el gateway empieza a enviarle tráfico: es el mismo mecanismo que se comprobó en local con dos instancias. La única excepción es el auth-server, que genera su par de claves al arrancar y debe correr con una sola tarea hasta que la clave se externalice.

6. Resultados obtenidos

6.1 Pruebas

Tabla 19: Pruebas unitarias

Módulo	Pruebas	Qué cubren
batch-migration	35	Formatos de fecha, mes 13, tildes, cálculo de interés, reglas de descarte y reejecución automática
banco-core-api	6	Retiros: saldo, montos, idempotencia
bff-web, bff-mobile, bff-atm	15	Composición de respuestas por canal y reglas del cajero
servicio-clientes	12	Validación del RUT
servicio-cuentas	6	Apertura, cliente inactivo, cierre con y sin saldo, cuenta cerrada
servicio-pagos	8	Saldo, cuenta bloqueada, alertas, idempotencia y orden de bloqueo
Total	82	

6.2 Evidencia de ejecución

Tabla 20: Resultados verificados en ejecución

Aspecto	Resultado
Migración batch	Tres jobs COMPLETED; 721 descartes con motivo; resultados idénticos con 1 y 4 particiones
Rendimiento del batch	36 % menos tiempo con cuatro particiones
Descubrimiento	Nueve servicios registrados en Eureka, servicio-pagos con dos instancias
Balanceo	Peticiones alternadas entre las dos instancias de pagos
Seguridad	401 sin token, 403 sin el scope, TLS 1.3 en el gateway
Dominio	Alta de cliente, apertura, depósito, transferencia, pago, bloqueo y cierre
Idempotencia	Una transferencia repetida no debitó dos veces ni generó un segundo aviso
Eventos y alertas	Avisos por cada operación y alertas por monto inusual y cuenta bloqueada
Resiliencia	Con clientes caído: 503 inmediato y circuito abierto; recuperación al volver

El detalle de cada resultado está en la carpeta `evidencias/09-eft` del repositorio, y se regenera con un script.

6.3 Comparación con el sistema legacy

Tabla 21: Sistema legacy frente al sistema nuevo

Aspecto	Legacy (COBOL y Shell en mainframe)	Sistema nuevo
Procesos batch	Scripts secuenciales sobre el mainframe	Jobs con steps, particiones configurables y reanudación desde el step fallido
Datos con errores	Sin trazabilidad declarada de lo que se descarta	Cada descarte registrado con su motivo; anomalías conservadas y marcadas
Fallas de infraestructura	El proceso se detiene y requiere intervención	Reintentos del chunk y reejecución automática
Escalabilidad	Vertical: más capacidad en el mismo equipo	Horizontal: cada servicio agrega instancias por separado
Canales	Todos reciben la misma respuesta	Un BFF por canal; el móvil transmite 89 % menos datos
Tolerancia a fallos	Un módulo con problemas afecta a todos	Circuit breaker y respuestas alternativas por dependencia
Seguridad	Centralizada	Token por servicio, permisos por scope, HTTPS y clave por terminal
Integración	Cada consumidor nuevo exige modificar el sistema	Eventos en tópicos: un consumidor nuevo se suscribe sin tocar a nadie
Configuración	Cambios con redesplicue	Configuración centralizada y versionable
Despliegue	Servidores mainframe	Contenedores, orquestación y procedimiento para AWS

7. Desafíos enfrentados y soluciones

7.1 Datos legacy con errores de todo tipo

Problema. Cuatro formatos de fecha en un mismo archivo, un mes que no existe, tipos con y sin tilde, montos vacíos y negativos.

Solución. Se perfilaron los archivos antes de escribir reglas. Las fechas se parsean en modo estricto para que el mes 13 falle en vez de corregirse en silencio, los textos se normalizan, y cada registro recibe un tratamiento explícito: válido, anomalía o descartado con su motivo.

7.2 Duplicados que caen en particiones distintas

Problema. El procesamiento en paralelo divide el archivo en rangos; dos filas idénticas pueden quedar en hilos distintos y ninguno ve a la otra.

Solución. La detección se movió a un step posterior a la carga, que compara con SQL sobre todo lo cargado, conserva la primera copia y registra las demás en la bitácora.

7.3 Filas de corridas anteriores mezcladas con las nuevas

Problema. Al cambiar de dataset, la tabla de transacciones tenía 956 filas cuando el archivo aportaba 789, y el resumen diario sumaba ambas.

Solución. El job pasó a recargar su resultado completo, igual que los otros dos. El error no se veía con el dataset anterior porque los identificadores coincidían.

7.4 El batch y los servicios comparten tablas

Problema. Si un servicio arrancaba antes que el batch sobre una base vacía, creaba las tablas sin las secuencias que el batch necesita. Además, el batch reconstruye la tabla de cuentas y borraría las abiertas en línea.

Solución. Se documentó que la carga inicial va primero, y las cuentas abiertas en línea se numeran desde 100.000, rango que el batch excluye de su reconstrucción. Se verificó que sobreviven a una reejecución.

7.5 Un circuit breaker que se abría antes de tiempo

Problema. Con el orden por defecto de los aspectos, cada reintento contaba como una falla aparte; y los rechazos de negocio, como saldo insuficiente, también contaban.

Solución. Una prueba con seis aperturas fallidas lo dejó en evidencia: el circuito registró 13 llamadas rechazadas, cifra que solo cuadra si cada reintento pasaba por él por separado. Se invirtió el orden de los aspectos para que el circuito envuelva al reintento y se repitió la prueba desde un circuito limpio: las mismas aperturas contaron como una llamada cada una. Los rechazos de negocio, además, se declararon como excepciones ignoradas.

7.6 Un circuito que quedaba abierto para siempre

Problema. Con la transición automática a semiabierto activada, Resilience4j 2.1.0 entraba en una recursión infinita y el circuito no volvía a cerrarse.

Solución. Se diagnosticó revisando la excepción real que recibía el método alternativo y se cambió a transición perezosa: el paso a semiabierto ocurre con la siguiente llamada. Se comprobó el ciclo completo cerrado, abierto, semiabierto y cerrado.

7.7 Concurrencia sobre el mismo saldo

Problema. Dos transferencias cruzadas pueden bloquearse mutuamente, y dos peticiones idénticas simultáneas podrían pasar ambas la verificación de idempotencia.

Solución. Las cuentas se bloquean siempre en el mismo orden, y la idempotencia se verifica después de tomar el bloqueo.

7.8 Llamadas entre servicios protegidas

Problema. servicio-cuentas necesita consultar a servicio-clientes, que exige token.

Solución. Se le dio su propio cliente OAuth2 con un único permiso de lectura, siguiendo el principio de mínimo privilegio.

7.9 El entorno de contenedores en Windows

Problema. Docker Desktop no levantaba su motor: el servicio quedaba detenido y, tras cada apagado, dejaba sockets huérfanos que el sistema no permitía borrar.

Solución. Se identificó la causa en los registros de Docker, se apartaron los sockets dañados y se desactivó el componente que los recreaba. El motor volvió a funcionar y la plataforma se ejecutó en contenedores.

8. Propuestas de mejora y próximos pasos

Tabla 22: Mejoras propuestas

Mejora	Qué resuelve	Plazo
Patrón outbox para los eventos	Hoy el evento sale después del commit; si el broker cae justo entre ambos, el aviso se pierde. Con outbox el evento se guarda en la misma transacción y otro proceso lo publica.	Corto
Clave del auth-server en Secrets Manager o KMS, con rotación	Permite escalar el servidor de autorización y que un reinicio no invalide los tokens vigentes.	Corto
Persistir notificaciones y alertas	Hoy se guardan en memoria; persistirlas y enviarlas por correo o SMS (Amazon SES, SNS) las vuelve útiles para el cliente.	Corto
Integración continua	Compilar, probar, construir imágenes y publicarlas en ECR en cada commit, con pruebas de integración sobre contenedores reales.	Corto
Una base de datos por servicio	Hoy la base es compartida en la etapa de transición. Separarla da independencia real a cada servicio; las operaciones entre servicios pasarían a coordinarse con sagas.	Mediano
Autenticación de usuarios finales	Agregar el flujo authorization_code con PKCE para que el cliente inicie sesión en la web y la app.	Mediano
Observabilidad centralizada	Prometheus y Grafana para tableros y alarmas, y un recolector de trazas (Zipkin o X-Ray) para ver el recorrido completo de una petición.	Mediano
Límite de peticiones en el gateway	Proteger los servicios ante abusos o picos, con un contador compartido en Redis y un WAF en el balanceador.	Mediano
Batch sobre almacenamiento en la nube	Leer los archivos desde S3 y usar particionamiento remoto para volúmenes que superen una sola máquina.	Largo
Cifrado entre servicios (mTLS)	Que cada servicio verifique la identidad del otro, no solo el token.	Largo

El siguiente paso concreto es el despliegue en AWS siguiendo `despliegue.md`: primero la opción con una instancia EC2 para validar el sistema en la nube, y luego la migración a ECS Fargate con base de datos administrada, una vez incorporados la integración continua y el patrón outbox.

9. Conclusiones

El proyecto reemplaza los procesos legacy del Banco XYZ por una plataforma que cumple los tres objetivos de la modernización. La migración batch procesa los archivos legacy con trazabilidad completa de lo que no entra, se recupera sola de fallas y entrega el mismo resultado en paralelo que en secuencia. Los BFF permiten que cada canal evolucione por su cuenta con la seguridad que le corresponde. Y los microservicios separan las responsabilidades del negocio en servicios que se descubren, se protegen, toleran fallas y se comunican por eventos.

Más allá de las tecnologías, la lección principal del proyecto fue que la mayoría de los problemas no estaban donde se esperaban: en datos que parecían limpios, en tablas compartidas, en el orden de dos anotaciones o en una operación concurrente. Verificar cada resultado con datos reales, y no solo con que el código compilara, fue lo que permitió encontrarlos.

Anexo A. Capturas de la plataforma en ejecución



HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-10-06T20:25:48 -0400
Data center	default	Uptime	00:23
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	38

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:api-gateway:8443
BANCO-CORE-API	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:banco-core-api:8080
BFF-ATM	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:bff-atm:8083
BFF-MOBILE	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:bff-mobile:8082
BFF-WEB	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:bff-web:8081
SERVICIO-CLIENTES	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:servicio-clientes:8085
SERVICIO-CUENTAS	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:servicio-cuentas:8086
SERVICIO-NOTIFICACIONES	n/a (1)	(1)	UP (1) - LAPTOP-2RGGN8RD:servicio-notificaciones:8084
SERVICIO-PAGOS	n/a (2)	(2)	UP (2) - LAPTOP-2RGGN8RD:servicio-pagos:8088 , LAPTOP-2RGGN8RD:servicio-pagos:8087

Figura 9. Registro de Eureka con los nueve servicios y dos instancias de servicio-pagos.

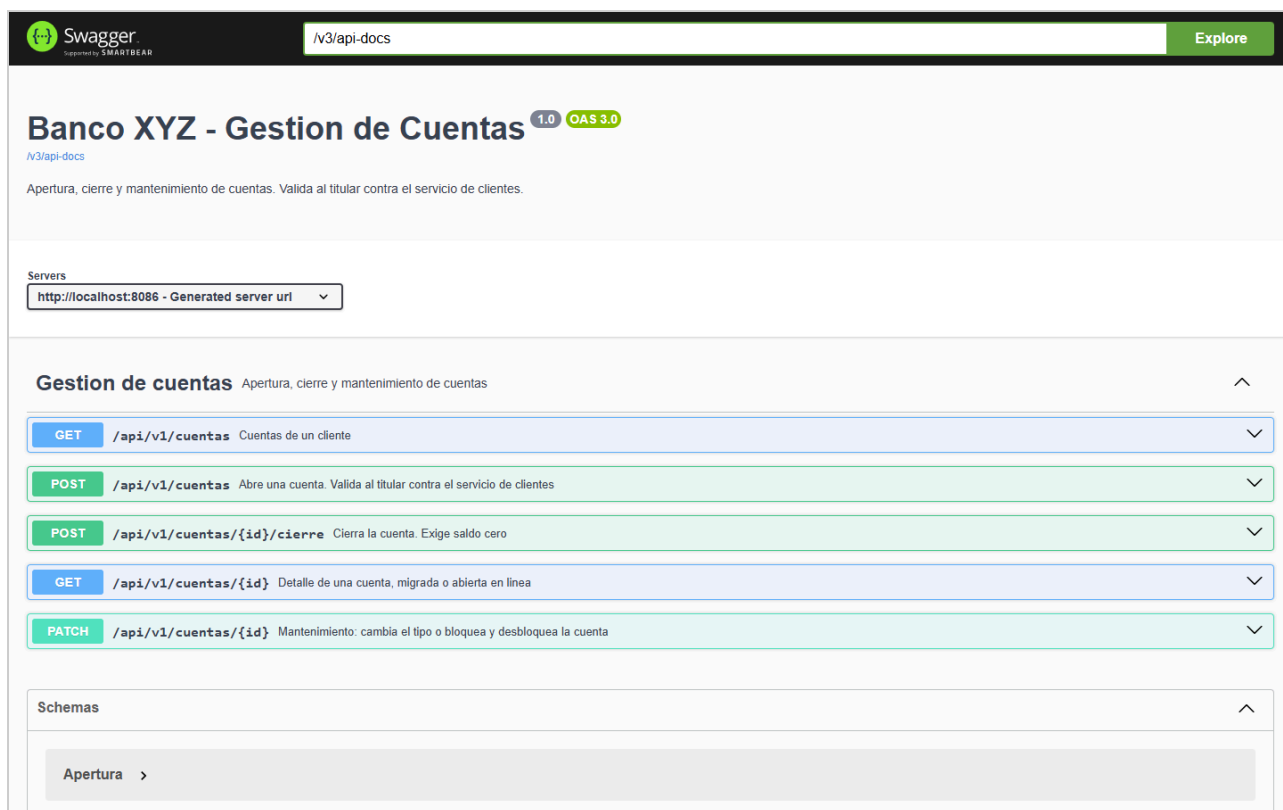


Figura 10. Documentación interactiva de servicio-cuentas.

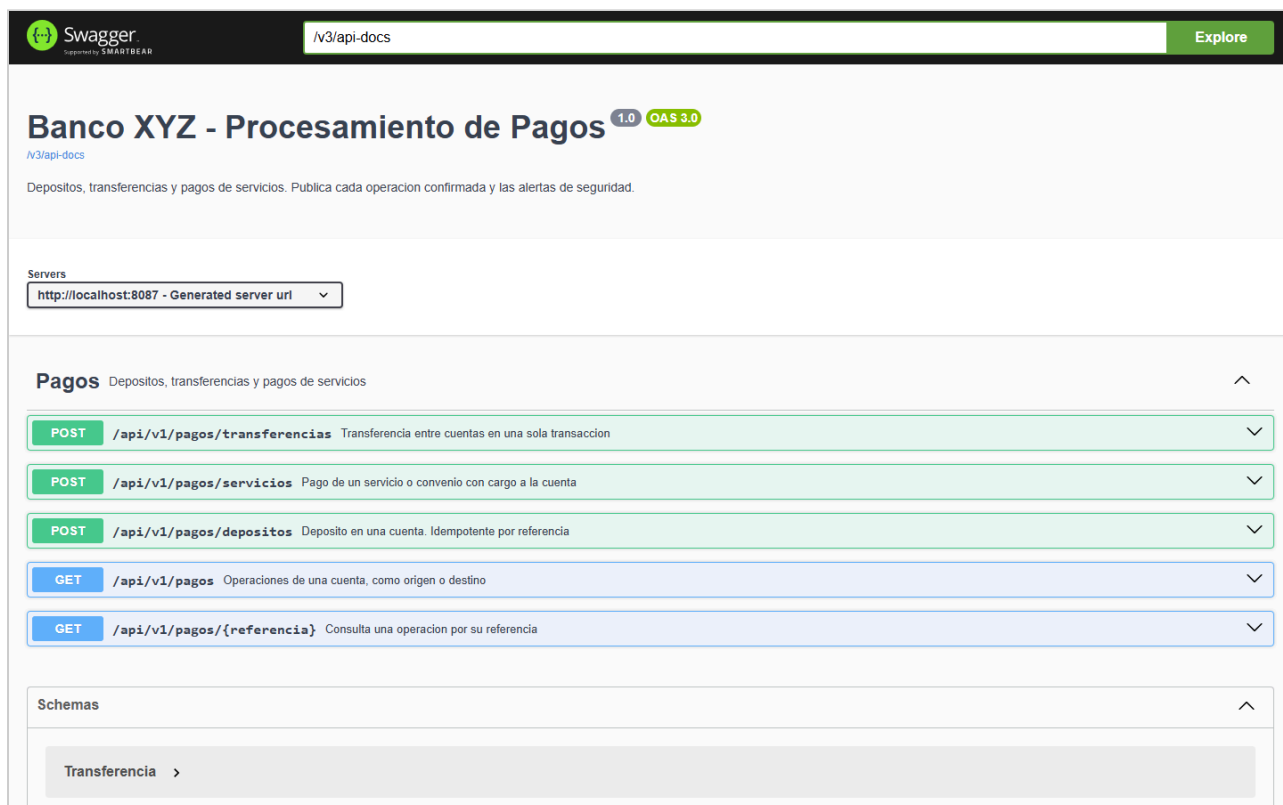


Figura 11. Documentación interactiva de servicio-pagos.

```

Pretty-print
[
  {
    "alertaId": "ALR-DEP-ALTO-202437",
    "tipo": "MONTO_INUSUAL",
    "severidad": "MEDIA",
    "cuentaId": 102,
    "monto": 1500000,
    "detalle": "operacion igual o superior al umbral de 1000000",
    "detectadaEn": "2026-10-06T20:24:41.2369961"
  },
  {
    "alertaId": "ALR-DEP-BLQ-202437-100003",
    "tipo": "CUENTA_NO_OPERATIVA",
    "severidad": "ALTA",
    "cuentaId": 100003,
    "monto": 5000,
    "detalle": "intento de operar sobre una cuenta en estado BLOQUEADA",
    "detectadaEn": "2026-10-06T20:24:41.1016372"
  },
  {
    "alertaId": "ALR-DEP-ALTO-202227",
    "tipo": "MONTO_INUSUAL",
    "severidad": "MEDIA",
    "cuentaId": 102,
    "monto": 1500000,
    "detalle": "operacion igual o superior al umbral de 1000000",
    "detectadaEn": "2026-10-06T20:22:40.3913567"
  }
]

```

Figura 12. Alertas de seguridad consumidas del tópico `banco.eventos.alertas`.

```

Pretty-print
{
  "circuitBreakers": {
    "clientes": {
      "failureRate": "80.0%",
      "slowCallRate": "0.0%",
      "failureRateThreshold": "50.0%",
      "slowCallRateThreshold": "100.0%",
      "bufferedCalls": 5,
      "failedCalls": 4,
      "slowCalls": 0,
      "slowFailedCalls": 0,
      "notPermittedCalls": 2,
      "state": "OPEN"
    }
  }
}

```

Figura 13. Estado del circuit breaker de servicio-cuentas con servicio-clientes detenido.

Anexo B. Entregables

Tabla 23: Ubicación de cada entregable

Entregable	Archivo
Código fuente	readme.md, con el enlace a github.com/JeanSantoni02/backend3
Informe técnico	Este documento
Instrucciones para ejecutar y probar	instrucciones.md
Pasos para desplegar en la nube	despliegue.md
Video de presentación	Archivo MP4 en la carpeta de entrega
Evidencia de ejecución	evidencias/09-eft en el repositorio